

AVS: A Computational and Hierarchical Storage System for Autonomous Vehicles

Yuxin Wang¹, Yuankai He¹, Damien Kah², and Weisong Shi¹

¹Department of Computer and Information Sciences, University of Delaware, Newark, DE 19716, USA

²Western Digital Corporation, San Jose, CA 95119, USA

Autonomous vehicles generate continuous multimodal sensor and state data, but most onboard systems still treat these data as transient inputs or short event records. As vehicles evolve into mobile computing platforms, this creates a storage gap for applications that depend on local historical data, including diagnostics, safety analysis, AI model improvement, and maintenance.

This digest presents Autonomous Vehicle Storage (AVS) as a design probe for this shift. AVS integrates utility-aware data reduction, append-only indexed logging, and an SSD–HDD hierarchy that keeps archived data queryable. It also supports two complementary offload patterns: selective network offload for critical records and physical HDD offload for bulk archive export. On a Raspberry Pi 5 prototype connected to a real Level 4 Lincoln MKZ, AVS reduces stored data by 8.0 to 8.7× compared with raw rosbag2. It supports hot-tier queries in 0.9 to 9.1 ms and cold-tier queries in 25.6 to 44.0 ms. It also recovers from power loss in 9.98 s with a bounded 0.24 s loss window. AVS points toward a vehicle oriented storage architecture and presents open challenges in building sustainable, queryable, and failure resilient onboard storage for heterogeneous long horizon vehicle workloads.

Index Terms—Autonomous vehicles, computational storage, hierarchical storage, vehicle computing.

I. MOTIVATION

AUTONOMOUS vehicles are entering the vehicle computing era [1], where onboard data support both real time driving and later data driven applications. During driving, sensor and vehicle state streams serve the driving stack and monitoring modules. After driving, the same data can support diagnostics, safety analysis, map maintenance, fleet learning, forensics, and third party services [2]. These applications need selected onboard history by time, modality, location, or event, rather than only live streams or full offline replay.

However, vehicle data are heterogeneous and at high rate. Cameras, LiDAR, and CAN bus streams can generate terabytes per day [3], while onboard capacity, flash endurance, power, thermal budget, and wireless offload remain limited. Existing event data recorders retain only narrow safety windows [4], and replay tools such as rosbag2 are designed for recording and playback rather than long horizon retention, selective query, or device efficient continuous ingest [5].

AVS explores a potential storage direction for this emerging vehicle data workload. First, computation is added during ingest to remove redundancy before it consumes storage capacity and device lifetime. Then, append-only logging is designed for continuous recording with later random and selective reading. And also, hierarchical storage is needed for low-latency access with long-horizon onboard retention. Through a concrete prototype, AVS exposes these design tradeoffs and identifies open challenges for vehicle oriented storage systems.

II. SYSTEM DESIGN

AVS follows a single forward data path. Streams arrive over Ethernet, pass through online computation, enter an SSD hot tier as indexed append logs, and later migrate to an HDD cold tier as indexed archives. Applications issue time and modality queries over both tiers using the same logical interface. Table I summarizes the design.

TABLE I
AVS DESIGN AND STORAGE REQUEST.

Layer	Mechanism	Vehicle Storage Request
Ingest compute	LiDAR voxel + LAZ; image pHash + JPEG; GNSS vector	Balance utility and latency, not only compression ratio
Hot tier	Trip append logs with chunk time indices	Support sequential writes and fast selective reads
Cold tier	Indexed tar archives on HDD	Make magnetic history random queryable, not only dumpable
Offload	Selective TCP or removable HDD export	Consistent query structure for export and post process
Recovery	Chunk validation and tail truncation	Robust recovery and bounded data loss

A. Utility aware ingest computation

AVS reduces data before persistence, focusing on LiDAR and camera, the two largest modalities. Ingest computation combines redundancy removal and compression, with each operator chosen by modality and downstream utility. For LiDAR, AVS voxel downsamples point clouds and compresses them with LAZ, keeping 24.38% of the original points, adding about 9 ms per scan, and further compressing the reduced cloud by 6.56× while preserving a representative localization workload. For camera streams, AVS removes redundant frames with perceptual hashing and stores the rest as independent JPEG 95 images, removing 28.25% of frames and compressing the remainder by 4.06× with 1.45 ms latency while preserving a representative perception workload.

B. Append logs as query structures

For each topic and trip, AVS writes a log file and a compact index. Records are grouped into chunks bounded by size or time. Each index entry stores start time, end time, file offset, byte size, and record count. A global SQLite catalog maps topic, day, and trip metadata to per trip indices. A query filters trips, searches the chunk index, and reads only matching byte ranges. For the same preprocessed camera, LiDAR, and GPS workload with 7,745 inserts and 1,000 range



Fig. 1. Lincoln MKZ Level 4 platform and AVS prototype. The AVS node is isolated from the driving computer and receives ROS 2 streams over Ethernet.

queries, the AVS append-only logger reduced insert latency to 0.171 ms, compared with 0.428 ms for SQLite and 0.569 ms for RocksDB, while keeping the smallest final storage size. This result also exposes a key design tradeoff: larger chunks improve write batching, while smaller chunks improve query selectivity and bound the recovery loss window after failures.

C. Queryable archival and offload

To match vehicle service and application usage, AVS supports both long horizon archival and practical data offload. Recent data stay on SSD for fast queries, while older completed topic day folders migrate to HDD as tar archives. Each archive includes a tar index with the byte offset and size of each embedded log and index file, so cold queries can read directly from HDD without unpacking the archive. This treats HDD as active historical memory, not only as backup storage. The same structure also enables two offload paths. For bandwidth constrained cases, AVS streams only selected query results over TCP. For bulk analysis, the HDD cold tier can be removed or mounted externally, since each archive carries its index and metadata database.

III. PROTOTYPE AND RESULTS

Figure 1 shows the prototype. AVS runs on a Raspberry Pi 5 with four Arm Cortex A76 cores, 8 GB RAM, a 256 GB NVMe SSD hot tier, and a 1 TB WD10SPZX HDD cold tier. Both tiers use XFS. The node runs Ubuntu 24.04 and ROS 2 Jazzy, and receives vehicle data through Ethernet. We evaluated three days of real Lincoln MKZ traces with concurrent 10 Hz Hesai Pandar64 LiDAR, 10 Hz front camera images, and 50 Hz NovAtel GNSS. AVS is compared with rosbag2 sqlite3 and mcap, with and without zstd, using the same streams.

Table II summarizes the main system results. The ingest results show that modality aware preprocessing reduces data more effectively than generic compression, but each operator must balance latency, downstream utility, and reduction ratio. They also show that even reduced vehicle data still creates sustained write pressure, making logging layout, media placement, and archival policy important for device lifetime. The memory result shows that AVS keeps bounded memory usage during concurrent multi sensor writing on the prototype platform. However, as sensor count and operator complexity grow, buffering and computation state may also grow. This ingest pattern raises an open question: how should vehicle storage reduce data early, preserve downstream utility, bound

TABLE II
PROTOTYPE RESULTS AND IMPLICATIONS.

Metric	Result	Implication
Final AVS size	23.4 to 25.4 GB per 66 to 70 min	Retains about one driving hour per day
Footprint vs. raw rosbag2	8.0 to 8.7 $\times$ lower	Reduces bytes before storage
Footprint vs. zstd rosbag2	4.5 to 5.0 $\times$ lower	Beats generic compression
SSD query TTFB	0.9 to 9.1 ms	Enables recent history inspection
HDD query TTFB	25.6 to 44.0 ms	Keeps HDD archives queryable
Recovery	9.98 s; 0.24 s loss	Bounds chunk loss
Archive and offload	107 MB/s; TCP/HDD	Supports retention and export
Memory	176 to 184 MB	Fits embedded storage

memory usage at scale, and extend media lifetime under sustained heterogeneous ingest?

The query and offload results show that hierarchical layout is important for vehicle storage. SSD keeps recent data available for frequent low latency inspection, while indexed HDD archives extend retention without losing selective queryability. The same indexed structure also makes offload more efficient and reusable: TCP export sends only requested records for bandwidth constrained cases, while physical HDD export moves self describing archives for bulk analysis. This usage pattern raises another open question: how should HDD based vehicle archives support fast archival writes, quick selective reads, and scalable bulk offload for large vehicle datasets?

Power loss recovery scans valid chunk boundaries, truncates the unsafe tail, repairs metadata, and resumes appending. In our prototype, recovery completes in 9.98 s with a bounded 0.24 s loss window. This deployment setting raises a challenge: how should vehicle storage bound data loss and recover quickly under unpredictable power failures while keeping computation, indexing, and archival overhead low?

IV. CONCLUSION

Vehicle computing is creating new storage needs for large, heterogeneous, and long-horizon onboard data. AVS shows that ingest computation and SSD plus HDD hierarchical storage are one possible direction, and it also shows the feasibility through a prototype deployed with a real autonomous vehicle platform. More importantly, it exposes more storage questions that need be addressed to make future vehicle storage systems more scalable, efficient, and feasible for practical deployment.

ACKNOWLEDGEMENT

This work is funded by the US National Science Foundation (NSF) under Grant CNS 2231523. Special thanks are extended to Western Digital for their support.

REFERENCES

- [1] S. Lu and W. Shi, "Vehicle computing: Vision and challenges," *Journal of Information and Intelligence*, vol. 1, no. 1, pp. 23–35, 2023.
- [2] F. Kazhamiaka, M. Zaharia, and P. Bailis, "Challenges and opportunities for autonomous vehicle query systems," in *Proc. CIDR*, 2021.
- [3] Y. Wang, Y. He, R. Wang, and W. Shi, "Quantitative analysis of storage requirement for autonomous vehicles," in *Proc. ACM HotStorage*, 2024, pp. 71–78.
- [4] National Highway Traffic Safety Administration, "Event data recorders," Final rule, 89 Federal Register 102810, Dec. 2024.
- [5] Open Robotics, "Recording and playing back data," ROS 2 Documentation, 2026.